Page-Agent + Gemini: Tutorial self-hosted setup

Czym jest page-agent?

Alibaba page-agent to biblioteka JavaScript, która osadza agenta AI bezpośrednio na stronie webowej. Agent odczytuje DOM strony, wysyła go do modelu językowego (LLM) i wykonuje akcje — klikanie przycisków, wypełnianie formularzy, nawigację — na podstawie poleceń w języku naturalnym. Wszystko dzieje się w przeglądarce, bez potrzeby rozszerzenia, headless browsera czy backendu.

Page-agent działa w modelu "Bring Your Own LLM" — możesz podłączyć dowolny model kompatybilny z API OpenAI: GPT, Claude, Qwen, Gemini (przez proxy) i inne.

Architektura rozwiązania

```
┌──────────────────────────────────────────────┐
│  Twoja strona HTML                           │
│                                              │
│  ┌──────────────────┐    ┌──────────────────┐  │
│  │  page-agent      │──►│ Cloudflare Worker  │  │
│  │  (bundle.js)     │◄──│ (proxy do Gemini)  │  │
│  └──────────────────┘    └──────────┬───────┘  │
│         │                           │          │
│         ▼                           ▼          │
│  ┌──────────────────┐    ┌──────────────────┐  │
│  │  Panel GUI       │    │ Google Gemini API  │  │
│  │  (wbudowany)     │    │ (REST endpoint)    │  │
│  └──────────────────┘    └──────────────────┘  │
└──────────────────────────────────────────────┘
```

Page-agent oczekuje API w formacie OpenAI (chat/completions). Gemini ma własny format REST API, dlatego potrzebny jest proxy (np. Cloudflare Worker), który tłumaczy zapytania między formatami.

Wymagania

- **Node.js** w wersji 20+ (`node --version`)
- **npm** (instaluje się z Node.js)
- **Przeglądarka** (Chrome, Firefox, Edge)
- **Cloudflare Worker** lub inny proxy tłumaczący format OpenAI → Gemini REST API
- **Klucz API Google Gemini** (z Google AI Studio: https://aistudio.google.com)

Krok 1: Przygotowanie katalogu projektu

Utwórz katalog roboczy i zainicjalizuj projekt npm:

```bash
mkdir page-agent-gemini
cd page-agent-gemini
npm init -y
```

Krok 2: Instalacja zależności

Zainstaluj page-agent, wymaganą zależność peer (zod) oraz bundler (esbuild):

```bash
npm install page-agent zod esbuild
```

Po instalacji w `node_modules/` pojawią się pakiety:

- `page-agent` — główna biblioteka z panelem UI
- `@page-agent/core` — headless core (logika agenta bez UI)
- `@page-agent/llms` — klient LLM z obsługą retry i patchami per model
- `@page-agent/ui` — komponent panelu i i18n
- `@page-agent/page-controller` — operacje na DOM
- `zod` — walidacja schematów (peer dependency)

Krok 3: Utworzenie pliku entry point

Utwórz plik `entry.js`, który importuje `PageAgent` i eksponuje go jako zmienną globalną na `window`:

```bash
echo 'import { PageAgent } from "page-agent"; window.PageAgent = PageAgent;' > entry
```

Zawartość pliku `entry.js`:

```javascript
import { PageAgent } from "page-agent";
window.PageAgent = PageAgent;
```

To jest jedyny kod potrzebny do bundlowania. Klasa `PageAgent` zawiera w sobie panel GUI, logikę agenta, klienta LLM i kontroler DOM.

Krok 4: Bundlowanie do pojedynczego pliku JS

Użyj esbuild, aby zbundlować wszystkie zależności w jeden plik IIFE (Immediately Invoked Function Expression):

```bash
npx esbuild entry.js --bundle --format=iife --outfile=page-agent.bundle.js --minify
```

Parametry:

- `--bundle` — włącza bundlowanie wszystkich importów
- `--format=iife` — generuje kod owinięty w natychmiast wywoływaną funkcję (działa bez modułów ES)
- `--outfile=page-agent.bundle.js` — nazwa pliku wyjściowego
- `--minify` — minifikuje kod (zmniejsza rozmiar z ~500KB do ~225KB)

Wynik:

```
  page-agent.bundle.js  225.1kb

⚡ Done in 186ms
```

Plik `page-agent.bundle.js` to samodzielny bundle — zawiera cały kod page-agent i jego zależności. Nie potrzebuje node_modules ani żadnych zewnętrznych zasobów.

Krok 5: Utworzenie strony HTML

Utwórz plik `index.html` w tym samym katalogu co `page-agent.bundle.js`:

```html
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>Page Agent - Gemini</title>
</head>
<body>

  <!-- Twoja strona / aplikacja / formularz do testowania -->

  <!-- Załaduj self-hosted bundle -->
  <script src="./page-agent.bundle.js"></script>

  <!-- Inicjalizacja agenta -->
  <script>
    window.addEventListener('DOMContentLoaded', () => {
      const agent = new PageAgent({
        model: 'gemini-3.1-flash-lite-preview',
        baseURL: 'https://TWOJ-WORKER.workers.dev/',
        apiKey: 'unused-by-proxy',
        language: 'en-US',
      });
      agent.panel.show();
    });
  </script>

</body>
</html>
```

Parametry konfiguracji PageAgent

Parametr	Opis
`model`	Identyfikator modelu Gemini (np. `gemini-3.1-flash-lite-preview`)
`baseURL`	URL Twojego proxy Cloudflare Worker (musi być kompatybilny z formatem OpenAI `/v1/chat/completions`)
`apiKey`	Klucz API — jeśli proxy sam dodaje klucz, wstaw dowolny placeholder
`language`	Język interfejsu panelu: `en-US`, `zh-CN`, `ja-JP`
`temperature`	Opcjonalnie, domyślnie `0.7`. Dla Gemini library automatycznie ustawia `reasoning_effort: minimal`
`maxRetries`	Opcjonalnie, domyślnie `2`. Ilość prób ponowienia przy błędach sieciowych

---

Krok 6: Testowanie

Otwórz `index.html` w przeglądarce. Możesz użyć prostego serwera HTTP:

```bash
# Python
python3 -m http.server 8080

# Node.js (npx)
npx serve .
```

Następnie otwórz `http://localhost:8080` w przeglądarce. W prawym dolnym rogu powinien pojawić się panel agenta.

Krok 7: Dodanie treści do testowania

Sam panel agenta na pustej stronie nie ma czego obsługiwać. Dodaj elementy UI, np. formularz:

```html
<body>
  <h1>Demo Page Agent + Gemini</h1>

  <form id="contact-form">
    <label>Imię: <input type="text" name="name" /></label><br><br>
    <label>Email: <input type="email" name="email" /></label><br><br>
    <label>Wiadomość: <textarea name="message" rows="4" cols="40"></textarea></label
    <button type="submit">Wyślij</button>
  </form>

  <script src="./page-agent.bundle.js"></script>
  <script>
    window.addEventListener('DOMContentLoaded', () => {
      const agent = new PageAgent({
        model: 'gemini-3.1-flash-lite-preview',
        baseURL: 'https://TWOJ-WORKER.workers.dev/',
        apiKey: 'unused-by-proxy',
        language: 'en-US',
      });
      agent.panel.show();
    });
  </script>
</body>
```

W panelu agenta wpisz np.: *"Fill the form with sample data"* i obserwuj jak agent wypełnia pola formularza.

Krok 8: Programowe wywołanie agenta

Oprócz panelu czatowego możesz wywoływać agenta z kodu JavaScript:

```javascript
// Wywołanie z przycisku
document.getElementById('demo-btn').addEventListener('click', () => {
  agent.execute('Fill the contact form with John Smith, john@example.com');
});

// Sterowanie panelem
agent.panel.show();      // pokaż panel
agent.panel.hide();      // ukryj panel
agent.panel.toggle();    // przełącz widoczność
agent.panel.reset();     // resetuj historię
agent.panel.dispose();   // usuń panel z DOM
```

Struktura plików projektu

```
page-agent-gemini/
├── index.html              ← Twoja strona
├── page-agent.bundle.js    ← Self-hosted bundle (jedyny plik JS potrzebny)
├── entry.js                ← Entry point do bundlowania (źródło)
├── package.json            ← Konfiguracja npm
├── package-lock.json
└── node_modules/           ← Zależności (nie commituj do repo)
```

Do repozytorium / hostingu wrzucasz tylko:

- `index.html`
- `page-agent.bundle.js`

Reszta (`node_modules/`, `entry.js`, `package.json`) to narzędzia deweloperskie do generowania bundle'a.

Aktualizacja page-agent w przyszłości

Sprawdzenie aktualnej i najnowszej wersji

```bash
# Twoja zainstalowana wersja
npm list page-agent

# Najnowsza dostępna na npm
npm view page-agent version

# Wszystkie dostępne wersje
npm view page-agent versions --json
```

Aktualizacja do najnowszej wersji

```bash
# Zaktualizuj paczkę
npm update page-agent

# LUB zainstaluj konkretną wersję
npm install page-agent@1.7.0

# Przebuduj bundle
npx esbuild entry.js --bundle --format=iife --outfile=page-agent.bundle.js --minify
```

Aktualizacja do konkretnej wersji

```bash
npm install page-agent@X.Y.Z
npx esbuild entry.js --bundle --format=iife --outfile=page-agent.bundle.js --minify
```

Skrypt automatyzujący

Dodaj do `package.json`:

```json
{
  "scripts": {
    "build": "esbuild entry.js --bundle --format=iife --outfile=page-agent.bundle.js
    "update": "npm update page-agent && npm run build"
  }
}
```

Wówczas aktualizacja sprowadza się do jednej komendy:

```bash
npm run update
```

Changelog i breaking changes

Przed aktualizacją sprawdź changelog na GitHubie:

```
https://github.com/alibaba/page-agent/releases
```

Page-agent stosuje semantic versioning:

- **Patch** (1.5.x → 1.5.y): poprawki bugów, bezpieczna aktualizacja
- **Minor** (1.5.x → 1.6.0): nowe funkcje, kompatybilne wstecz
- **Major** (1.x → 2.0): breaking changes, sprawdź migrację

Alternatywa: użycie demo.js z URL params (bez bundlowania)

Jeśli nie potrzebujesz programowej kontroli nad agentem, możesz skopiować plik demo bezpośrednio z paczki npm:

```bash
npm install page-agent
cp node_modules/page-agent/dist/iife/page-agent.demo.js ./page-agent.demo.js
```

I użyć go z konfiguracją w parametrach URL:

```html
<script src="./page-agent.demo.js?model=gemini-3.1-flash-lite-preview&baseURL=https:
  crossorigin="true">
</script>
```

Demo.js automatycznie tworzy instancję `PageAgent`, pokazuje panel i jest gotowy do użycia. Nie masz jednak kontroli nad instancją agenta z kodu JavaScript.

Rozwiązywanie problemów

Problem	Przyczyna	Rozwiązanie
Panel nie pojawia się	Bundle nie załadowany	Sprawdź ścieżkę do `page-agent.bundle.js` w konsoli przeglądarki (F12 → Network)
Błąd CORS	Proxy nie zwraca nagłówków CORS	Dodaj `Access-Control-Allow-Origin: *` w Cloudflare Worker
Agent nie odpowiada	Błąd API	Sprawdź konsolę (F12 → Console) — logi LLM pojawią się tam
"LLM configuration required"	Brak wymaganych pól	Upewnij się, że `model`, `baseURL` i `apiKey` są podane
401/403 z API	Nieprawidłowy klucz	Sprawdź klucz API Gemini i konfigurację proxy
Agent nie wykonuje akcji	Model za słaby	Gemini Flash Lite wymaga minimum 15k tokenów kontekstu. Upewnij się, że strona nie jest zbyt złożona

---

Przydatne linki

- **Repozytorium page-agent:** https://github.com/alibaba/page-agent
- **Dokumentacja:** https://alibaba.github.io/page-agent/docs/introduction/overview
- **npm:** https://www.npmjs.com/package/page-agent
- **Google AI Studio (klucze Gemini):** https://aistudio.google.com
- **Gemini API docs:** https://ai.google.dev/docs